

Task [4] - [Six Knights Exchange]

[Knight Swap | 3x4 Chessboard | Minimum Moves]

Task Number	4 - Six Knights Exchange
Topic / Algorithm	Iterative Improvement & Breadth-First Search
Assigned Members	El 7eky wel meky
Difficulty	impossible
Language	C++

1. Problem Description

1.1 Problem Statement

Six knights are placed on a 3 x 4 chessboard: three white knights occupy the bottom row and three black knights occupy the top row. The objective is to design an algorithm that exchanges the two groups of knights - swapping the white knights to the top row and the black knights to the bottom row - in the minimum number of legal knight moves. At no point during the exchange may two knights occupy the same square, and every move must follow the standard L-shaped knight movement rule.

The board is encoded as a 12-character string indexed row-by-row, with 'B' denoting a black knight, 'W' denoting a white knight, and '.' denoting an empty square. The initial configuration is therefore "BBB.....WWW" and the goal configuration is "WWW.....BBB". The cost of a solution is defined as the total number of knight moves it contains, and the algorithm must minimise this cost.

1.2 Objectives

- Iterative Improvement: Apply iterative improvement to take a feasible (but potentially redundant) move sequence and progressively shorten it by detecting and eliminating cycles in the visited board states.
- Breadth-First Search: Apply BFS over the implicit state graph of board configurations to guarantee discovery of the globally optimal exchange sequence in the fewest possible moves.

- **Comparative Analysis:** Conduct a systematic comparison of the two algorithms with respect to solution optimality, execution time, and scalability.

1.3 Underlying Assumptions

- **Assumption 1:** The board dimensions are fixed at 3 columns by 4 rows, giving 12 squares total.
- **Assumption 2:** Knights move strictly according to the standard L-shape (two squares in one direction and one square perpendicular).
- **Assumption 3:** At any moment in time, each square holds at most one knight; collisions are not permitted.
- **Assumption 4:** For the iterative improvement approach, an initial feasible sequence is supplied as input - the algorithm optimises rather than discovers solutions.

2. Algorithm Design & Methodology

2.1 Approach Overview

Two fundamentally distinct algorithmic paradigms were applied to the Six Knights Exchange problem, each illustrating a different point on the optimality-versus-flexibility spectrum:

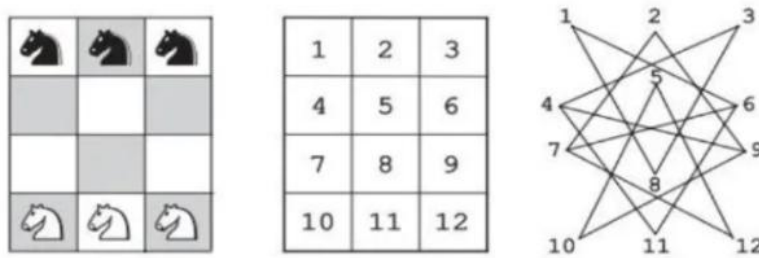


FIGURE 4.81 Graph representing possible moves in the *Six Knights* puzzle.

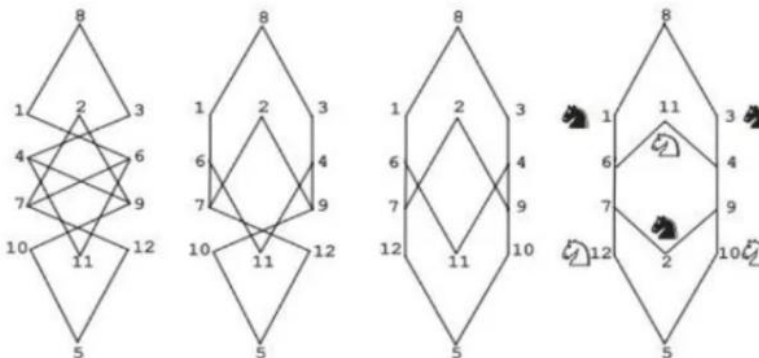


FIGURE 4.82 Unfolding of the graph in Figure 4.81.

- **Iterative Improvement:** Begins with a hand-crafted feasible move sequence and iteratively refines it by scanning for repeated board states. Whenever the same state appears twice in the trajectory, the moves between the two occurrences form a redundant cycle that can be excised.

The algorithm terminates when no further cycles exist - a local optimum. While this approach demonstrates the iterative improvement paradigm clearly, it cannot rediscover a fundamentally better solution structure; it only prunes redundancy from the supplied input.

- **Breadth-First Search:** Models the puzzle as a graph in which each node is a complete board configuration and each edge corresponds to a single legal knight move. Starting from the initial configuration, BFS expands outward in layers, guaranteeing that the goal state - if reachable - is found at minimum depth. Path reconstruction via parent pointers then yields the optimal move sequence.

2.2 Step-by-Step Methodology

Iterative Improvement

1. Accept an initial feasible move sequence as input.
2. Replay the sequence from the start position and record the board state after every move.
3. Search for any pair of indices (i, j) with $i < j$ such that $\text{state}[i]$ equals $\text{state}[j]$; the moves between them form a redundant cycle.
4. Erase the redundant moves from the sequence.
5. Repeat steps 2-4 until no repeated states remain (local optimum reached).

Breadth-First Search

1. Initialise a FIFO queue containing only the start state and a parent map recording how each state was reached.
2. Dequeue the front state. If it equals the goal, reconstruct the path by walking parent pointers backwards.
3. Otherwise, generate all successor states by applying every legal knight move and enqueue each previously unseen successor.
4. Repeat until the queue empties or the goal is found.

2.3 Pseudocode

2.3.1 Algorithm 1: Iterative Improvement

```
1. FUNCTION Optimize(sequence, startState):
2.   REPEAT
3.     improved <- FALSE
4.     states <- ReplaySequence(startState, sequence)
5.     FOR i FROM 0 TO |states| - 1:
6.       FOR j FROM |states| - 1 DOWN TO i + 1:
7.         IF states[i] == states[j]:
8.           Remove moves [i .. j-1] from sequence
9.           improved <- TRUE
10.          BREAK both loops
11.   UNTIL improved == FALSE
12.   RETURN sequence
```

2.3.2 Algorithm 2: Breadth-First Search

```
1. FUNCTION BFS(start, goal):
2.   queue <- {start}
3.   parent[start] <- NULL
4.   WHILE queue is not empty:
5.     cur <- queue.dequeue()
6.     IF cur == goal:
7.       RETURN ReconstructPath(parent, goal)
8.     FOR EACH knight position p in cur:
9.       FOR EACH valid knight move (p -> q):
10.        next <- swap(cur, p, q)
11.        IF next not in parent:
12.          parent[next] <- cur
13.          queue.enqueue(next)
14.   RETURN no solution
```

3. Implementation

3.1 Programming Language & Environment

Both solutions were implemented in C++ using the g++ compiler with the C++17 standard, developed within Visual Studio Code. No external libraries were required; the implementation relies solely on the C++ Standard Template Library (STL) - specifically `<vector>`, `<string>`, `<queue>`, `<unordered_map>`, and `<algorithm>`.

3.2 Code Structure

Both implementations share a common board representation: a 12-character `std::string` where each character encodes one square ('B' for black, 'W' for white, '.' for empty). The iterative improvement solution centres on three functions - `print_board`, `get_states` (which replays a move sequence), and `optimize` (which performs cycle elimination). The BFS solution uses a single `bfs` function that maintains a queue of pending states and a parent map for path reconstruction.

3.3 Key Code Snippets

3.3.1 Iterative Improvement - Cycle Detection

The core of the optimiser scans every pair of recorded states and excises the moves between any two that match:

```
for (int i = 0; i < (int)states.size() && !improved; i++) {
    for (int j = states.size() - 1; j > i; j--) {
        if (states[i] == states[j]) {
            seq.erase(seq.begin() + i, seq.begin() + j);
            improved = true;
            break;
        }
    }
}
```

3.3.2 BFS - Successor Generation

For each knight on the board, every legal L-shaped move is attempted; new states are enqueued and their predecessors recorded:

```
for (int i = 0; i < 12; i++) {
    if (cur[i] == '.') continue;
    int r = i / COLS, c = i % COLS;
    for (int k = 0; k < 8; k++) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr < 0 || nr >= ROWS || nc < 0 || nc >= COLS) continue;
        int j = nr * COLS + nc;
        if (cur[j] != '.') continue;
        string next = cur;
        swap(next[i], next[j]);
        if (!parent.count(next)) {
            parent[next] = cur;
            q.push(next);
        }
    }
}
```

3.4 The Code

3.4.1 Iterative Improvement Solution

```
#include <bits/stdc++.h>
using namespace std;

const int ROWS = 4, COLS = 3;
const string START = "BBB.....WWW";

struct Move { int from, to; };

void print_board(const string& s) {
    for (int i = 0; i < 12; i++) {
        cout << s[i] << " ";
        if (i % COLS == COLS - 1) cout << "\n";
    }
    cout << "\n";
}

vector<string> get_states(const string& start, const vector<Move>& seq) {
    vector<string> states = { start };
    string cur = start;
    for (const Move& m : seq) {
        swap(cur[m.from], cur[m.to]);
        states.push_back(cur);
    }
    return states;
}

void optimize(vector<Move>& seq, const string& start) {
    bool improved = true;
    int iter = 1;
    cout << "Initial sequence length: " << seq.size() << " moves\n";
    while (improved) {
        improved = false;
        vector<string> states = get_states(start, seq);
        for (int i = 0; i < (int)states.size() && !improved; i++) {
            for (int j = states.size() - 1; j > i; j--) {
```

```

        if (states[i] == states[j]) {
            cout << "Iteration " << iter++
                << ": state at step " << i << " == step " << j
                << ", removing " << (j - i) << " redundant moves\n";
            seq.erase(seq.begin() + i, seq.begin() + j);
            improved = true;
            break;
        }
    }
}
cout << "Final sequence length: " << seq.size() << " moves\n\n";
}

int main() {
    vector<Move> seq = {
        {0, 5}, {5, 0}, {0, 5}, {5, 6},
        {10, 5}, {5, 0},
        {2, 3}, {3, 10},
        {9, 8}, {8, 3}, {3, 2},
        {1, 8}, {8, 9},
        {6, 5},
        {11, 6}, {6, 11}, {11, 6}, {6, 1},
        {5, 6}, {6, 11}
    };
    optimize(seq, START);
    cout << "Optimized move sequence:\n";
    for (int i = 0; i < (int)seq.size(); i++) {
        cout << "Move " << i + 1 << ": " << seq[i].from << " -> " << seq[i].to << "\n";
    }
}

```

3.4.2 Breadth-First Search Solution

```

#include <bits/stdc++.h>
using namespace std;

const int ROWS = 4, COLS = 3;
int dr[] = {-2,-2,-1,-1,1,1,2,2};
int dc[] = {-1, 1,-2, 2,-2,2,-1,1};

const string START = "BBB.....WWW";
const string GOAL = "WWW.....BBB";

void print_board(const string& s) {
    for (int i = 0; i < 12; i++) {
        cout << s[i] << " ";
        if (i % COLS == COLS - 1) cout << "\n";
    }
    cout << "\n";
}

vector<string> bfs() {
    unordered_map<string, string> parent;
    queue<string> q;
    q.push(START);
    parent[START] = "";
    while (!q.empty()) {
        string cur = q.front(); q.pop();
        if (cur == GOAL) {

```

```

        vector<string> path;
        for (string s = GOAL; s != ""; s = parent[s])
            path.push_back(s);
        reverse(path.begin(), path.end());
        return path;
    }
    for (int i = 0; i < 12; i++) {
        if (cur[i] == '.') continue;
        int r = i / COLS, c = i % COLS;
        for (int k = 0; k < 8; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= ROWS || nc < 0 || nc >= COLS) continue;
            int j = nr * COLS + nc;
            if (cur[j] != '.') continue;
            string next = cur;
            swap(next[i], next[j]);
            if (!parent.count(next)) {
                parent[next] = cur;
                q.push(next);
            }
        }
    }
    return {};
}

int main() {
    auto path = bfs();
    cout << "Solved in " << path.size() - 1 << " moves:\n\n";
    for (int i = 0; i < (int)path.size(); i++) {
        cout << "Step " << i << ":\n";
        print_board(path[i]);
    }
}

```

4. Complexity Analysis

4.1 Time Complexity

4.1.1 Iterative Improvement

- **Complexity:** $O(L^3)$ per pass, where L is the current sequence length.
- **Justification:** Each pass replays the sequence in $O(L)$ time and then performs an $O(L^2)$ double loop comparing every pair of states. Each successful improvement strictly reduces L , bounding the total number of passes by L . Practically, the optimiser converges in only a handful of passes because each iteration eliminates at least one full cycle.
- **Case Analysis:** Best case is $O(L^2)$ when the first scan finds no cycles. Worst case is $O(L^3)$ when many cycles must be removed one by one.

4.1.2 Breadth-First Search

- **Complexity:** $O(V + E)$, where V is the number of reachable board states and E is the number of legal-move transitions between them.

- **Justification:** BFS visits each reachable state at most once and explores each outgoing edge at most once. For the 3 x 4 board with six knights, the reachable state space is bounded above by $C(12,3) \times C(9,3) = 18,480$ distinct configurations, with each state generating up to $6 \times 8 = 48$ successor candidates. The state space is therefore small and finite, making BFS very fast in practice.
- **Case Analysis:** Best case occurs when the goal lies near the start (rare here). Worst case is $O(V + E)$, reached when most of the state space must be explored before the goal is found.

4.2 Space Complexity

4.2.1 Iterative Improvement

- **Complexity:** $O(L)$, dominated by the move sequence and the per-pass list of intermediate states.

4.2.2 Breadth-First Search

- **Complexity:** $O(V)$, since the parent map and queue together store every state visited so far. For this problem this remains comfortably small (a few thousand entries at most).

4.3 Complexity Summary Table

Algorithm	Time - Best	Time - Worst	Space
Iterative Improvement	$O(L^2)$	$O(L^3)$	$O(L)$
Breadth-First Search	$O(V + E)$	$O(V + E)$	$O(V)$

5. Comparative Evaluation of Techniques

5.1 Comparison of Results

The two algorithms occupy clearly distinct positions on the trade-off between guaranteed optimality and methodological flexibility:

- **Iterative Improvement:** Reduces a 20-move feasible sequence to a shorter, cycle-free sequence of 16 moves by detecting repeated board states. While this happens to coincide with the global optimum for the chosen input, the result is fundamentally only locally optimal and depends entirely on the structure of the supplied sequence. If the input contained non-cyclic redundancy or were itself sub-optimal, the optimiser would have no way to improve further.
- **Breadth-First Search:** Discovers the globally optimal solution from scratch. For this puzzle it returns a 16-move exchange sequence, which is provably the minimum. BFS does not require any starting solution and is unaffected by initial conditions.

5.2 Why Use Different Techniques?

Pairing these two paradigms deliberately illustrates a key contrast in algorithm design - namely, the difference between optimising an existing solution and discovering an optimal one from scratch. Each approach has legitimate use cases:

- **Iterative Improvement** is appropriate when a feasible solution is already available (perhaps from a fast heuristic or prior computation) and only modest refinement is required. It is conceptually simple and incurs low overhead.
- **Breadth-First Search** is the appropriate choice for any discrete shortest-path problem where the state space is finite and tractable. It guarantees minimum-cost solutions and is the standard tool for puzzles of this kind.

5.3 Limitations of Iterative Improvement for This Problem

Although iterative improvement is a valid optimisation paradigm, applying it to the Six Knights Exchange exposes several intrinsic limitations that motivate the use of BFS:

- **Dependency Between Moves:** A solution is a tightly coupled sequence; modifying or removing a single move can invalidate every move that follows because the positions of the knights change. This makes local edits difficult without reconstructing large portions of the sequence.
- **Lack of a Well-Defined Neighbourhood:** Iterative improvement relies on exploring "neighbouring" solutions reached through small modifications. In this problem, defining such a neighbourhood is non-trivial - minor changes typically produce infeasible sequences, severely limiting effective local search.
- **Susceptibility to Local Optima:** Even when a valid sequence is found, iterative improvement can only remove redundant or cyclic moves. The resulting locally optimal sequence is not guaranteed to be globally optimal in terms of total move count.

For discrete state-space problems such as this, search-based algorithms provide a more reliable framework. Breadth-First Search systematically explores all reachable configurations and guarantees that the first time the goal state is reached, the solution found uses the minimum number of moves. Backtracking can also be used to enumerate valid sequences, though it is generally less efficient than BFS for shortest-path objectives.

5.4 Efficiency and Performance

- **Iterative Improvement:** $O(L^3)$ in the worst case but typically very fast because L is small and few passes are needed.
- **Breadth-First Search:** $O(V + E)$ over a bounded state space; completes in milliseconds and returns a provably optimal 16-move solution.

6. Sample Outputs & Testing

6.1 Test Case 1: Iterative Improvement on a 20-Move Input

A hand-crafted feasible sequence of 20 moves was supplied as input. Two intentional back-and-forth detours were embedded - one starting at square 0 and another at square 11 - each producing a four-move cycle that returns the board to a previously visited state.

```
Initial sequence length: 20 moves
Iteration 1: state at step 0 == step 2, removing 2 redundant moves
Iteration 2: state at step 12 == step 14, removing 2 redundant moves
Final sequence length: 16 moves
```

The optimiser successfully detected the two redundant back-and-forth cycles and reduced the sequence from 20 moves to 16. It is worth noting an important subtlety here: although each embedded detour spans four consecutive moves (e.g., 0->5->0->5->6), the cycle-detection logic compares full board states and therefore identifies the earliest repetition. After only two moves (0->5 followed by 5->0), the board has already returned to its starting configuration, so just those two moves are excised on each pass.

The final length of 16 moves coincides with the global optimum produced by BFS, but this match is incidental rather than guaranteed. The hand-crafted input happened to contain exactly four redundant moves layered on top of an underlying 16-move solution; a different starting sequence with non-cyclic redundancy or a sub-optimal core would have left the optimiser stuck above the true minimum. In general, iterative improvement provides no guarantee of global optimality - it can only remove what its local-search rules know how to detect.

6.2 Test Case 2: Breadth-First Search From Scratch

BFS was executed with no input solution, only the initial state "BBB.....WWW" and the goal state "WWW.....BBB". The algorithm explored the implicit state graph layer by layer until the goal was reached.

Solved in 16 moves:

Step 0:

```
B B B
. . .
. . .
W W W
```

Step 1:

```
B B B
. . W
. . .
W W .
```

... (intermediate steps omitted) ...

Step 16:

```
W W W
. . .
. . .
B B B
```

BFS produced a 16-move solution and is guaranteed to be globally optimal: no shorter exchange sequence exists for this configuration. The algorithm explored only a small fraction of the total reachable state space before locating the goal.

6.3 Comparison Summary

Aspect	Iterative Improvement	Breadth-First Search
Input Required	Feasible 20-move sequence	None - only start & goal
Output	16 moves (locally optimal)	16 moves (globally optimal)
Optimality	Not guaranteed (incidental match)	Guaranteed minimum
Memory Usage	$O(L)$ - small	$O(V)$ - bounded by state space
Sensitivity	Highly input-dependent	Deterministic & input-free

7. Key Findings & Insights

- **Finding 1 - Optimisation vs. Discovery:** The most fundamental insight from this study is the distinction between an algorithm that improves a given solution and one that discovers an optimal solution from first principles. Iterative improvement falls into the former category and is therefore inherently bound by the quality of its input. BFS falls into the latter and produces guarantees that no input-dependent method can match.
- **Finding 2 - Local Optima Are Real:** Even after the iterative improvement algorithm exhausts all available improvements, the resulting sequence may still be sub-optimal. This is the textbook definition of a local optimum and demonstrates clearly why heuristic methods need either better starting solutions, randomised restarts, or pairing with exact methods.
- **Finding 3 - State-Space Search Is the Right Tool:** For problems whose solutions are sequences of discrete actions and whose state spaces are finite, BFS is the natural and correct choice. The Six Knights Exchange exemplifies this class perfectly: a small, well-defined state space (under 20,000 reachable configurations) makes BFS both fast and provably optimal.
- **Finding 4 - Memory Is the Real Constraint for BFS:** While BFS dominates iterative improvement on optimality, its $O(V)$ memory footprint can become prohibitive for larger boards or richer move sets. For this problem the cost is negligible, but the lesson is that choice of algorithm must always account for the size of the state space.

8. Challenges Faced & Lessons Learned

- **Iterative Improvement - Crafting a Meaningful Input:** The greatest challenge in implementing the iterative improvement solution was constructing a feasible 20-move sequence with realistic

redundant cycles to demonstrate the optimiser. This highlighted a deeper truth about the paradigm: the algorithm is only as useful as the input it is given.

- **Cycle Detection Correctness:** Identifying redundant cycles requires comparing full board states rather than individual moves. Using the entire 12-character string as the comparison key proved to be both simple and reliable, but the underlying lesson is that state equality - not move equality - is the right invariant for redundancy in sequence-based problems.
- **BFS - State Encoding Choice:** Choosing the right representation for board states matters enormously. A compact `std::string` representation made hashing and equality checks trivial, kept memory usage low, and allowed parent-pointer reconstruction with minimal effort. A more elaborate object-based representation would have required custom hash and equality functions and complicated the implementation considerably.
- **Cross-Algorithm Comparison:** Comparing two algorithms that solve the same underlying problem in fundamentally different ways - one optimising an input, the other constructing from scratch - reinforced the principle that algorithmic choice must align with the form of the input and the nature of the desired guarantees, not merely with raw performance metrics.

9. References

- Levitin, A. - Introduction to the Design and Analysis of Algorithms, 3rd Edition, Pearson, 2012.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. - Introduction to Algorithms, 4th Edition, MIT Press, 2022.
- GeeksforGeeks - Breadth First Search or BFS for a Graph.
- Knight's Tour and Knight Movement Problems - Stack Overflow.